Real-time Demo Workshop Rev. 3

(With additional exercises)

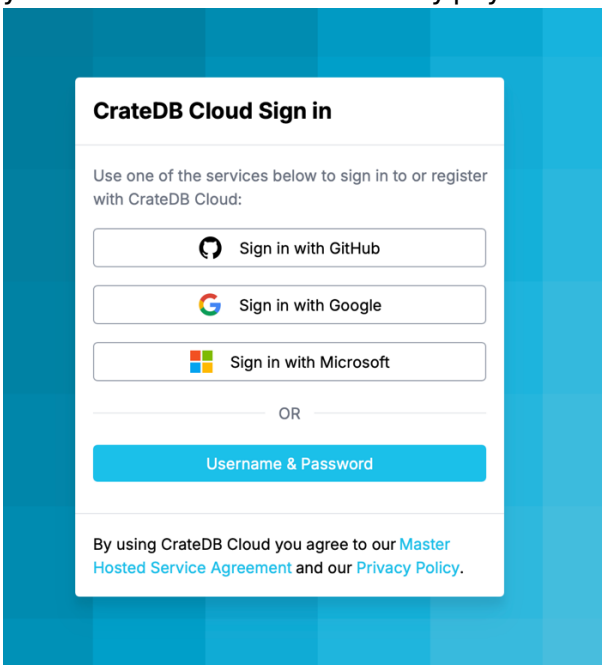
CrateDB

Table of Contents

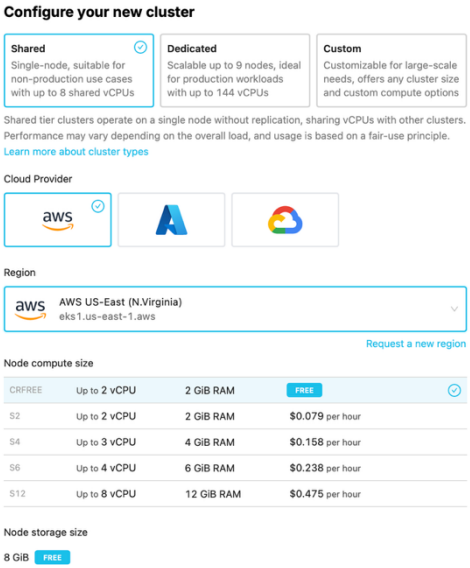
Workshop Setup Instructions

Creating a CrateDB Cluster

1. Go to https://cratedb.com
2. Click on the Start Free button
3. You have several options to create an account, use the one that is appropriate for you. You won't need to enter any payment details.



4. Once you have an account, log in and deploy a CRFREE cluster



   - Select a Shared cluster (this is the only way to get a free cluster)
   - Select AWS as the cloud provider
   - Select AWS US-East as the region (the same as the workshop runs within)

- Ensure CRFREE is selected as compute size
- Click Deploy Cluster to create it, which will take a short amount of time

5. It's very important that you save the login credentials, so download or make a note now before moving on.
6. We need to create the destination table for the climate data, run the following in the Console:

```
CREATE TABLE IF NOT EXISTS "demo"."climate_data" (
   "timestamp" TIMESTAMP WITHOUT TIME ZONE,
   "geo_location" GEO_POINT,
   "data" OBJECT(DYNAMIC) AS (
      "temperature" DOUBLE PRECISION,
      "u10" DOUBLE PRECISION,
      "v10" DOUBLE PRECISION,
      "pressure" DOUBLE PRECISION,
      "latitude" DOUBLE PRECISION,
      "longitude" DOUBLE PRECISION
   )
);
```

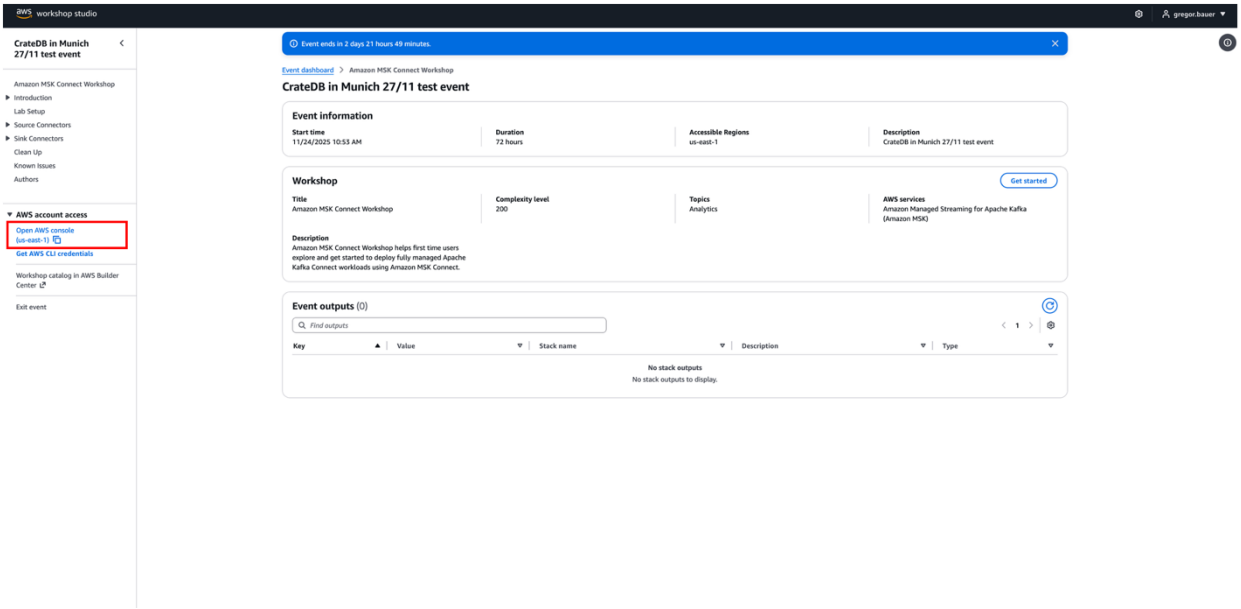
7. The host information you will need later, so make a note or leave the webpage open for now.

Creating AWS Workshop Pre-Requisites

Note: Please use the exact names provided in this guide for naming instances otherwise build steps might fail. Use copy paste to avoid errors.

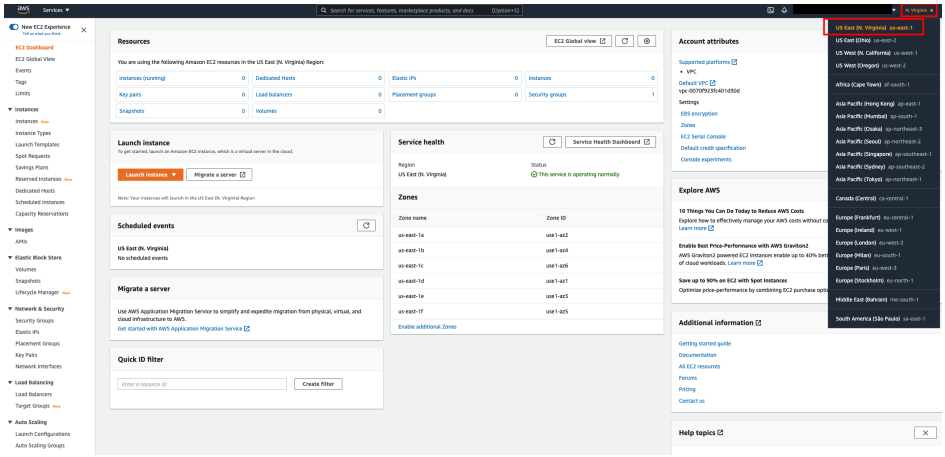
Note: If you already have AWS console it's best to use another browser

1. Click on the provided link and add the email you registered for the event to get an OTP
2. After entering the OTP, you received go to the Amazon Console

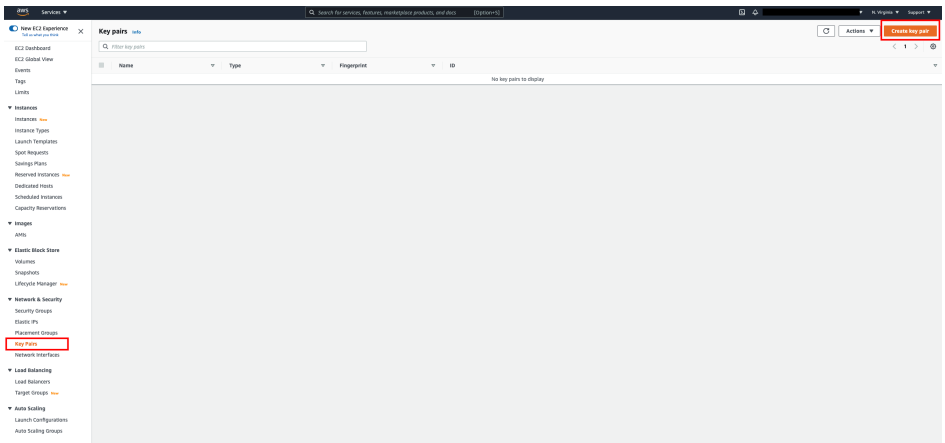


3. Open the Amazon EC2 console at https://console.aws.amazon.com/ec2/

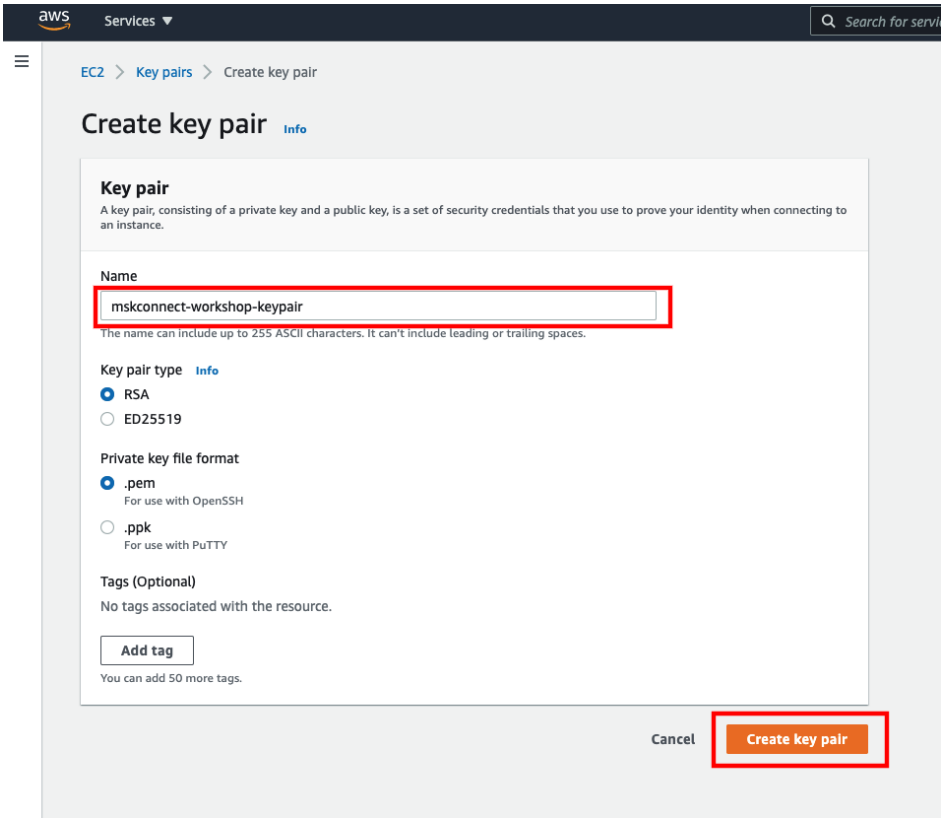
Choose region us-east-1 this will be the region that you are using for the workshop.



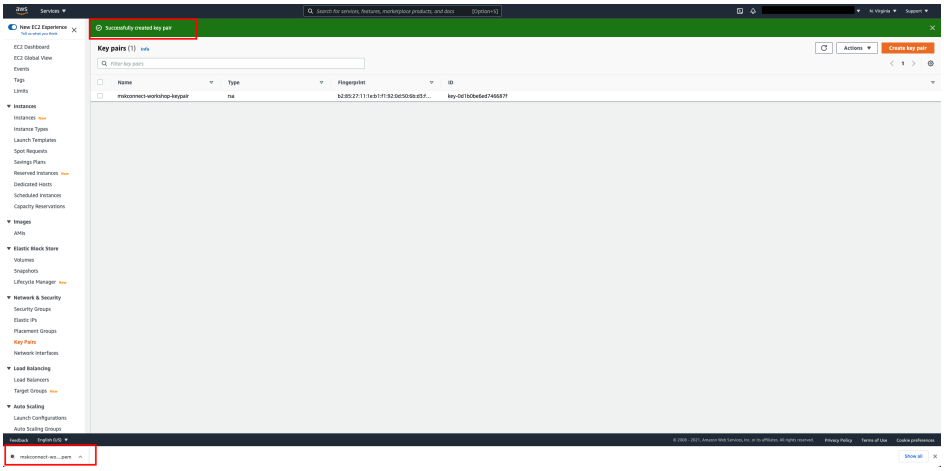
2. Choose Key Pairs in the navigation bar on the left and click on Create key pair.



3. Provide a name for the key pair (e.g. mskconnect-workshop-keypair) and click on Create key pair.

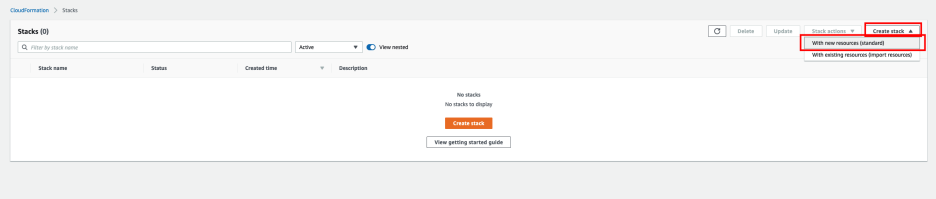


4. You should see the message Successfully created key pair, and confirm the download of the generated .pem file to your local machine.

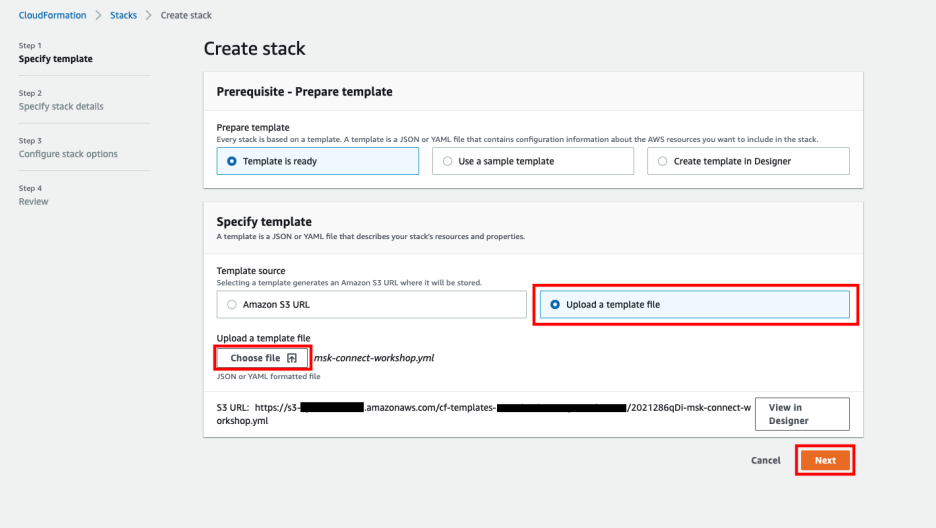


5. Download the CloudFormation template here: https://github.com/crate/realtime-demo/releases/download/CloudFormation/CloudFormationTemplateWithLambdaV2.yaml (you may need to copy/paste rather than click the link in this PDF, remember there are NO spaces)

6. Open the AWS CloudFormation console at https://console.aws.amazon.com/cloudformation/
7. Click on Create stack and select With new resources (standard).



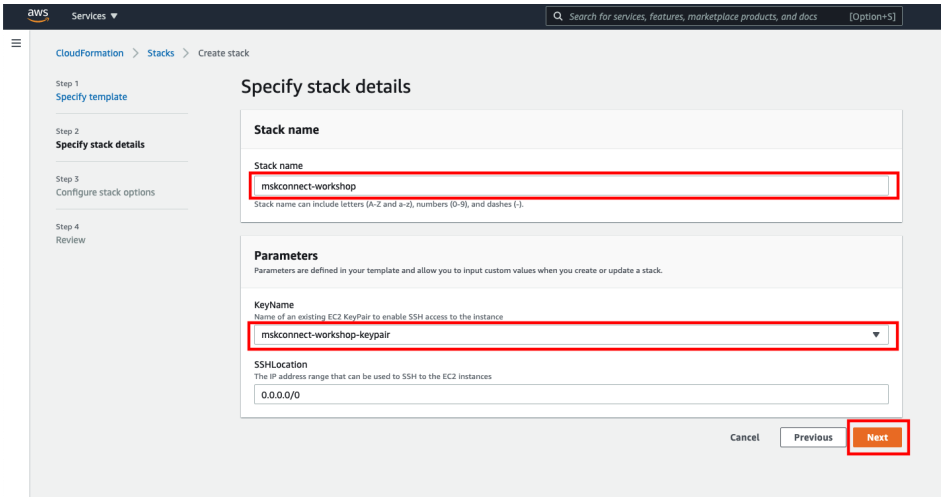
8. Choose Upload a template file option and upload the CloudFormation template from your local machine, then click Next.



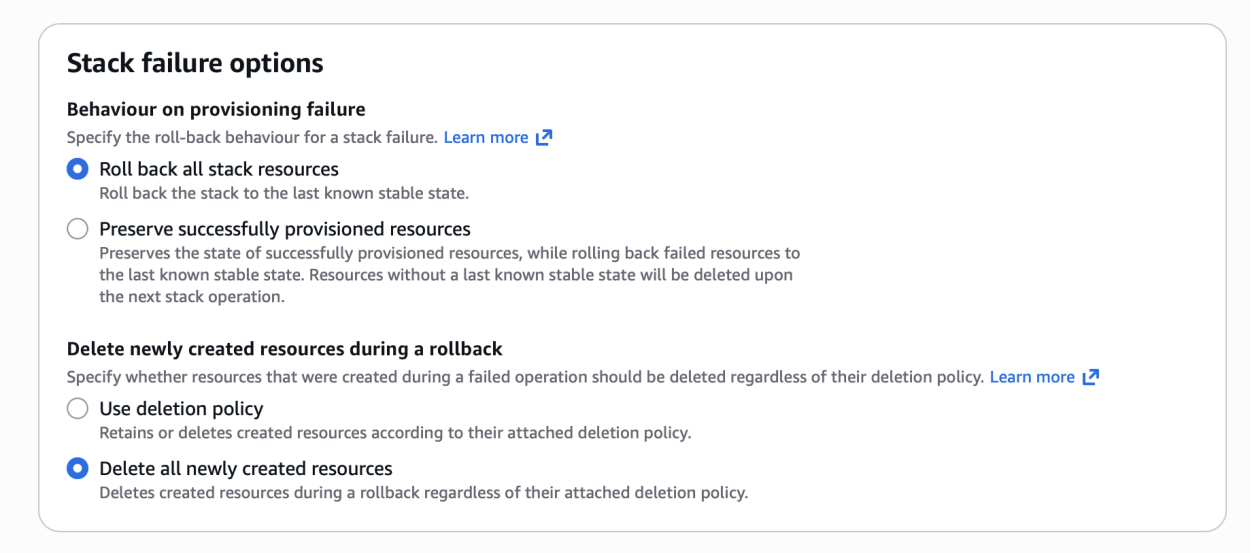
9. Enter mskconnect-workshop as the stack name. There are parameters on the next screen that need to entered correctly, although some have sensible default values (and likely won't need to be changed). Each parameter has a description to help identify what it needs to be configured as but ask for help if unsure.

   Choose mskconnect-workshop-keypair under KeyName.

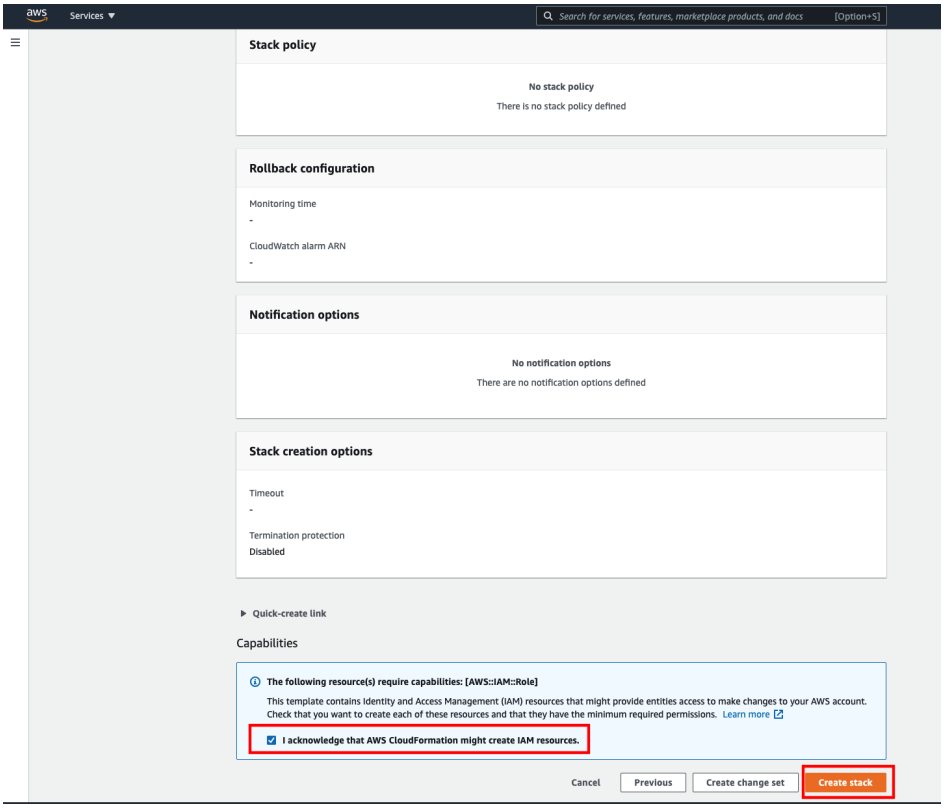
   Then click Next.



10. For the step Configure stack options, ensure Delete newly created resources during a rollback is set to Delete all newly created resources.

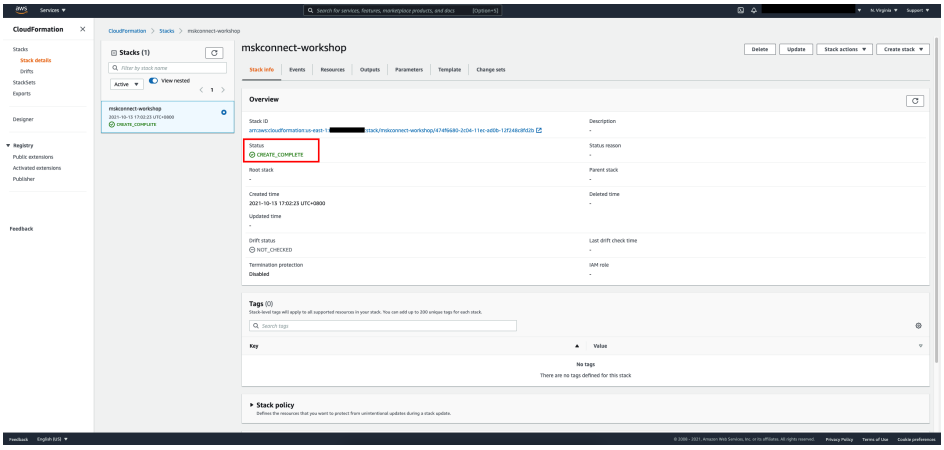


Scroll down to the bottom and check the box to acknowledge the creation of IAM resources.



Click Next and then Submit to trigger the creation of the CloudFormation stack.

12. Wait until the stack creation is completed before proceeding to the next steps. It may take a while for the MSK cluster creation (~30 to 45 mins).



What does the above CloudFormation template do?

The CloudFormation stack creates the following resources:

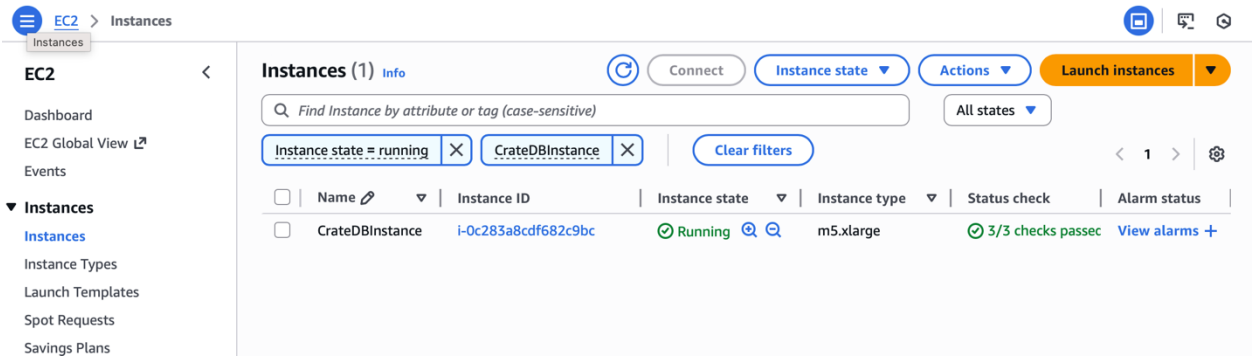
- 2 CloudWatch Log Groups for MSK Cluster and MSK Connect connectors respectively, each with 7 days log retention period
- 1 S3 bucket
- 1 VPC with 1 Public subnet and 3 Private subnets, and the required networking components such as Route Tables, Internet Gateway, NAT Gateway & Elastic IP, VPC Gateway Endpoint for S3
- 1 m5.large EC2 instance with the required Security Group, IAM Role, EC2 Instance Profile. On this instance various things will be installed including Python 3.13, netCDF4 and other libraries needed to execute the data producer.
- 1 MSK Cluster with 3 kafka.m5.large broker nodes configured with Apache Kafka version 3.6.0, and the required Security Group

Note that the encryption options are disabled to reduce the chance of errors in the workshop, this is not a best practice, and you should not use the provided CloudFormation template in any production environments.

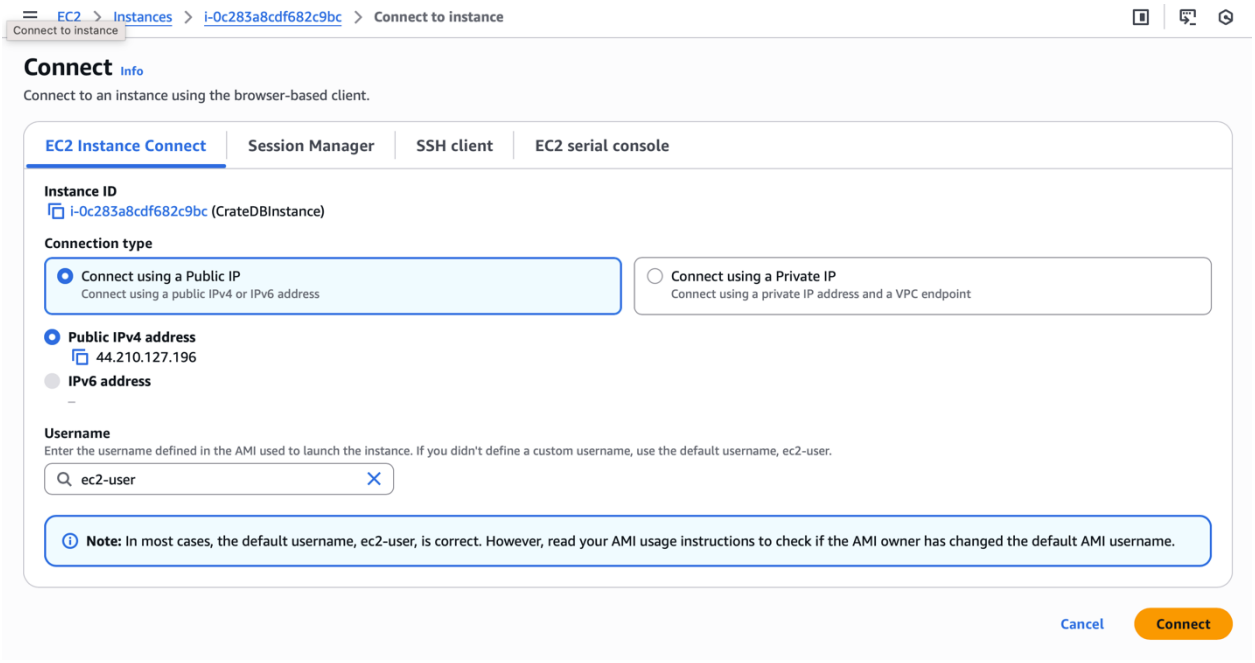
Checking EC2 installation progress

Open the EC2 dashboard https://console.aws.amazon.com/ec2/ and go to Instances

Wait for the instance (named CrateDBInstance) to be fully ready before attempting to connect (it will show state as running).



Select it and click on the Connect button.



In the shell use the following command to follow progress:

```
tail -f /var/log/user-data.log
```

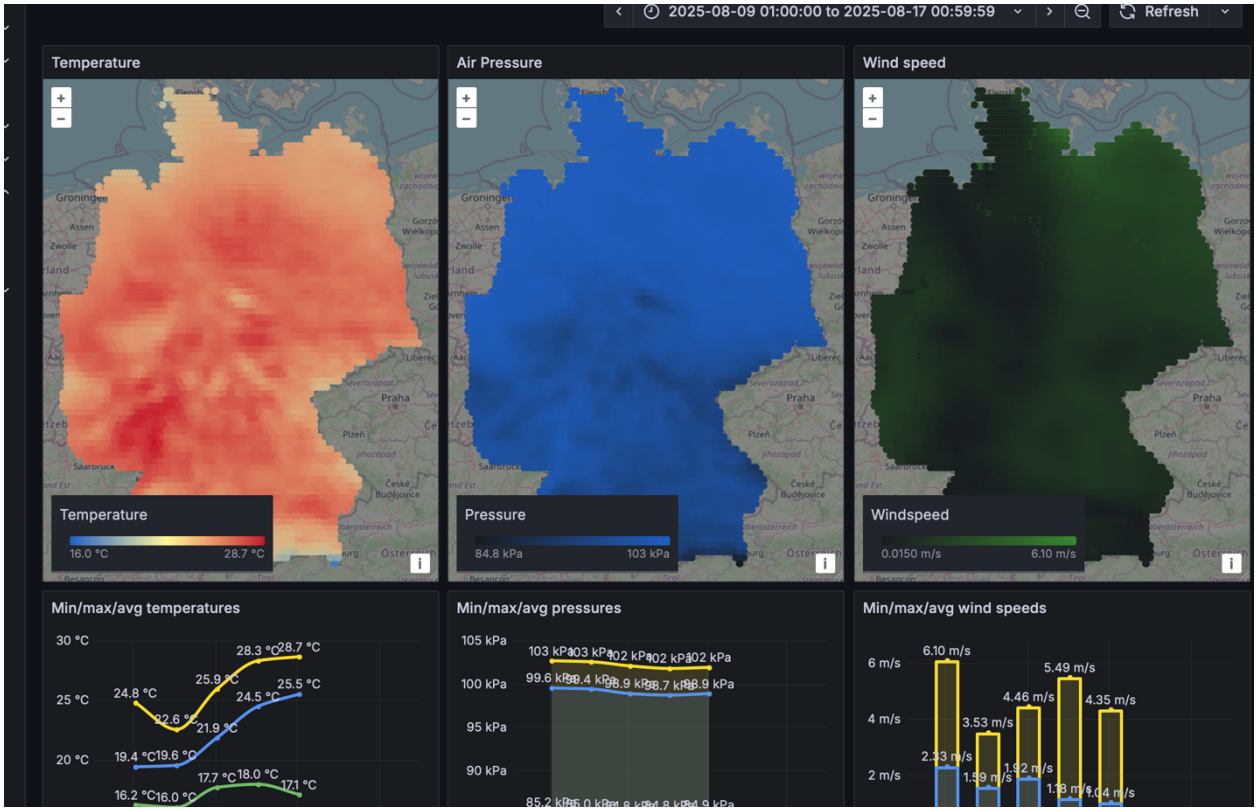
Wait until the following is displayed (note this may take some time):

Everything is installed!

View Grafana dashboards

Your dashboard should be accessible at http://ip:3000 (where IP is the assigned external IP address of the EC2 instance).

To login, the default Grafana username and passwords are admin/admin, then either skip or set a new password.



All complete!

Re-running the data producer

The data producer will run during the installation, but if you want to run the demo again and want to see the visualization in real time building up, simply go back to your query console and truncate the table:

```
delete from demo.climate_data;
```

Then run the following from the EC2 instance:

```
cd ~/realtime-demo/data
source .venv/bin/activate
python3 producer.py
```

Change country for climate data

If you like, you can modify the source data to be local to the Netherlands. It will require to get an account for the weather data and the data to be downloaded by the producer code which can take some time. Here are the steps of how to do it.

Change Country to Netherlands (NLD)

In your parser code initialization, change:

```
parser = ClimateParser("DEU")
```

to:

```
parser = ClimateParser("NLD")
```

NOTE: Clear Old Data (delete table) BEFORE starting the producer

Delete the existing data so the dashboard shows only fresh Netherlands data:

```
delete from demo.climate_data;
```

Start the Producer Again

Now run the producer to download and ingest new data:

```
python3 producer.py
```

The pipeline retrieves ERA5-Land data for the Netherlands, parses it, and produces JSON records.

View Updated Grafana Dashboard

Open Grafana and refresh the dashboard.

It will now display only the newly ingested Netherlands data.

Additional Exercises

Try some queries to see the flexibility in the data model

What is the time range?

```
SELECT
  min("timestamp") AS start_ts,
  max("timestamp") AS end_ts
FROM
  demo.climate_data;
```

Timestamps in CrateDB are always stored without a timezone. If the field is defined WITH a timezone, then on ingest the timestamp is adjusted accordingly to UTC.

In this data, it's defined without a timezone. It's defined as the number of milliseconds since the Unix epoch (`1970-01-01T00:00:00Z`). For a detailed guide to all the available datatypes see:

https://cratedb.com/docs/crate/reference/en/latest/general/ddl/data-types.html#timestamp

Note: The CrateDB console will display both the number of milliseconds and the human readable date and time.

Distribution of temperature

```
SELECT
  min(data['temperature']) AS min_t,
  avg(data['temperature']) AS avg_t,
  max(data['temperature']) AS max_t
FROM
  demo.climate_data;
```

This is a straightforward aggregation query. It's running a query across all data in the table and returning the minimum, maximum and average temperatures.

Notice the `AS`, which is an alias. Normally fields in an aggregate will be unnamed expressions, but with recent versions of CrateDB it will generate an alias automatically. However, it's still recommended to give a name.

Average temperature per day

```
SELECT
  date_trunc('day', "timestamp") AS day,
  min(data['temperature']) AS min_temp,
  max(data['temperature']) AS max_temp
FROM
  demo.climate_data
GROUP BY
  1
ORDER BY
  1;
```

This query introduces a new function - `date_trunc`. This returns a timestamp value *truncated* to a specific precision, in this case to precision of a day and effectively ignores the HH:MM:SS of any timestamp. This gives us a per-day aggregate figure. To get information on more functions like this, see:

https://cratedb.com/docs/crate/reference/en/latest/general/builtins/scalar-functions.html

10 hottest points

```
SELECT
  "timestamp",
  geo_location,
  data['temperature'] AS temp
FROM
  demo.climate_data
ORDER BY
  temp DESC
LIMIT
  10;
```

To give us the top 10 hottest points, we use `ORDER BY` (to order by temperature in descending order) and `LIMIT` to only return 10 rows.

The `SELECT` part of the query returns us the timestamp (i.e. when), the location (as a `GEO_POINT` co-ordinate, which is explained later) and the temperature from the `data` JSON object.

Are we missing any data points?

```
SELECT
  count(*) FILTER (WHERE data['temperature'] IS NULL) AS missing_temp,
  count(*) FILTER (WHERE data['pressure'] IS NULL) AS missing_pressure,
  count(*) FILTER (WHERE geo_location IS NULL) AS missing_geo
FROM
  demo.climate_data;
```

As you can see, nothing is missing in the included data. How does this query work?

`FILTER` is a window function. It performs a calculation across a set of related rows defined by an expression (or multiple expressions). In this case we are `COUNT`ing the number of rows where one of `data['temperature']`/`data['pressure']` or `geo_location` is NULL (i.e. undefined).

Window functions are very powerful, and it's very much recommended to read the following webpage to learn more about their capabilities.

https://cratedb.com/docs/crate/reference/en/latest/general/builtins/window-functions.html

Let's add a point with a missing temperature and re-run:

```
INSERT INTO demo.climate_data (timestamp, geo_location, data) VALUES
(1754956800000, [8.78831111111111, 54.903], {"longitude" = 8.78831111111111,
"latitude" = 54.903, "u10" = 4.472952365875244, "v10" = -1.3958832025527954,
"pressure" = 102426.1015625});
```

If you now re-run the missing data point query, it will show:

```
missing_temp missing_pressure    missing_geo
1            0                   0
```

Now let's update the record with a valid temperature:

```
UPDATE demo.climate_data
SET
  data['temperature'] = 16.8
WHERE
  timestamp = 1754956800000
  AND geo_location = [8.78831111111111, 54.903];
```

Re-run the missing data point query, and you'll see it now shows zero again.

Wind speed calculations

This query will calculate min/max and avg wind speeds (in metres per second) using Pythagoras' theorem to calculate the diagonal (hypotenuse) value:

$\sqrt{(x^2 + y^2)}$

```
SELECT
  min(sqrt(power(data['u10'], 2) + power(data['v10'], 2))) AS min_ws,
  avg(sqrt(power(data['u10'], 2) + power(data['v10'], 2))) AS avg_ws,
  max(sqrt(power(data['u10'], 2) + power(data['v10'], 2))) AS max_ws
FROM demo.climate_data;
```

The `power` function returns the field to the specified power, 2 in this case, i.e. it squares the field value. The `sqrt` functions returns the square root. These are part of the CrateDB scalar functions, and more information is available here:

https://cratedb.com/docs/crate/reference/en/latest/general/builtins/scalar-functions.html

Can we optimise this query? The mathematical expression is being calculated 3 times and this is inefficient. We can use a CTE (common table expression) to optimise this:

```
WITH wind_speed AS (
    SELECT
        sqrt(power(data['u10'], 2) + power(data['v10'], 2)) AS ws
    FROM demo.climate_data
)
SELECT
    MIN(ws) AS min_ws,
    AVG(ws) AS avg_ws,
    MAX(ws) AS max_ws
FROM wind_speed;
```

Now we are only calculating once. For more information on common table expressions, see:

https://cratedb.com/querying/common-table-expressions-ctes

What are the top 10 windiest observations?

```
SELECT
  "timestamp",
  geo_location,
  sqrt(power(data['u10'], 2) + power(data['v10'], 2)) AS wind_speed
FROM
  demo.climate_data
ORDER BY
  wind_speed DESC
LIMIT 10;
```

This uses the wind speed calculation as detailed in the previous query.

Wind direction calculated using trigonometry

This query will show the wind direction in degrees.

```
SELECT
  date_trunc('hour', "timestamp") AS hour,
  avg(sqrt(power(data['u10'], 2) + power(data['v10'], 2))) AS avg_speed,
  avg(
    mod(degrees(atan2(data['u10'], data['v10'])) + 360, 360)
  ) AS avg_dir_deg
FROM demo.climate_data
GROUP BY 1
ORDER BY 1;
```

The function `mod` has been introduced here, this refers to the `modulus` operation (`mod` is a shortened alias to `modulus`). This function returns the remainder of a divide operation; in this case we use it so that the angle in degrees is always between 0 and 360, i.e. an angle of 370 would return 10.

The other new function is `atan2`, which is a trigonometry function. All these are covered in the scalar functions guide:

https://cratedb.com/docs/crate/reference/en/latest/general/builtins/scalar-functions.html#scalar-modulus

Exercise: try returning the result in radians (ensure you update the modulo operation, the `pi` function will be useful here!)

CrateDB

Exercises - Geo

How to extract latitude & longitude from a GEO_POINT

You can use the `latitude` and `longitude` functions to return the co-ordinates in a `GEO_POINT` datatype. This example will return the bounding box for all loaded data:

```
SELECT
  min(longitude(geo_location)) AS min_latitude,
  max(longitude(geo_location)) AS max_latitude,
  min(latitude(geo_location)) AS min_latitude,
  max(latitude(geo_location)) AS max_latitude
FROM
  demo.climate_data;
```

Let's use the result of this to work out how many data points there are in a subset of the data, take this query and fill in some correct values based on the result of the above query.

```
SELECT count(*) AS points
FROM demo.climate_data
WHERE latitude(geo_location) BETWEEN 0.0 AND 0.0
  AND longitude(geo_location) BETWEEN 0.0 AND 0.0;
```

If you get it right, this should return a non-zero count. If it's zero, check your query and try again.

More information on CrateDB's geospatial capabilities is detailed here:

https://cratedb.com/docs/guide/start/modelling/geospatial.html

A more powerful way of doing this is to use the `within` function. This allows us to find points that lie within a polygon, which gives us the flexibility to use shapes other than a rectangle:

```
SELECT count(*) AS points
FROM demo.climate_data
WHERE within(
    geo_location,
    {
      "type": "Polygon",
      "coordinates": [[
        [0.0, 0.0],
        [0.0, 1.0],
        [1.0, 1.0],
        [1.0, 0.0],
        [0.0, 0.0]
      ]]
    }
);
```

The list of points is an array, in the form:

[longitude1, latitude1],

[longitude2, latitude2] ,

[longitude3, latitude3] ,

[longitude4, latitude4] ,

[longitude1, latitude1]

Which effectively *defines* a rectangular bounding box.

Note: Don't be afraid to ask for some guidance from one of the CrateDB team.

Distance calculations

CrateDB also supports distance calculations, the following query gives the nearest points to Berlin, Germany (approximate location is 52.5N, 13.4E)

```
SELECT
  geo_location,
  distance(geo_location, 'POINT(13.4 52.5)') AS metres,
  data['temperature'] AS temp
FROM demo.climate_data
GROUP BY geo_location, metres, temp
ORDER BY metres ASC
LIMIT 50;
```

Exercise: Try changing the location from Berlin to some other location.

Exercises – Correlation

Is pressure and temperate related?

Is there a relationship between temperature and atmospheric pressure (at ground level)?

Run this query and decide for yourself:

```
SELECT
  geo_location,
  avg(data['temperature']) AS avg_temp,
  avg(data['pressure']) AS avg_pressure
FROM demo.climate_data
GROUP BY 1
ORDER BY 2 DESC;
```

Exercise: Is there a way to improve this query? Maybe a combination of temperature variance from average and air pressure from 100,000 (1 atmosphere)?

Is pressure and wind speed related?

Is there a relationship between temperature and wind speed?

```
SELECT
  geo_location,
  avg(data['temperature']) AS avg_temp,
  avg(data['pressure']) AS avg_pressure,
  avg(sqrt(power(data['u10'], 2) + power(data['v10'], 2))) AS wind_speed
FROM demo.climate_data
GROUP BY 1
ORDER BY 3 DESC;
```

Exercise: Try altering the ordering, either **ASC**ending or **DESC**ending, or using a different column (or combination).

Exercise – JSON object model

Insert a new value into the dataset.

```
INSERT INTO demo.climate_data (timestamp, geo_location, data) VALUES
(123, [8.78831111111111, 54.903], {"longitude" = 8.78831111111111,
"latitude" = 54.903, "temperature" = 16.868310546875023, "u10" =
4.472952365875244, "v10" = -1.3958832025527954, "pressure" =
102426.1015625});
```

This is using the existing schema. What if we want to add a new humidity value to our dataset, do we need to recreate the table?

In CrateDB we can define object columns (i.e. JSON) as one of three different types:

`OBJECT(STRICT)` – The schema for the object is fixed
`OBJECT(DYNAMIC)` – The schema is dynamic, and new attributes can be added
`OBJECT(IGNORED)` – The JSON is stored ignoring the datatypes of the attributes and not indexing the field

https://cratedb.com/blog/handling-dynamic-objects-in-cratedb

When you created this table you set the `data` column to be DYNAMIC, so we can add additional attributes easily, and these attributes will be indexed.

Add a new key in the JSON object without the need to modify the schema

```
INSERT INTO demo.climate_data (timestamp, geo_location, data) VALUES
(123, [8.78831111111111, 54.903], {"humidity" = 43.4, "longitude" =
8.78831111111111, "latitude" = 54.903, "temperature" =
16.868310546875023, "u10" = 4.472952365875244, "v10" = -
1.3958832025527954, "pressure" = 102426.1015625});
```

Now try to query the new key:

```
SELECT
  data['humidity']
FROM
  demo.climate_data
WHERE
  data['humidity'] IS NOT NULL;
```

This makes it very easy to extend the data model, ideal for IoT applications where additional sensor information may become available.

And of course, with CrateDB all of this additional data will be indexed for great query performance.

Exercise: Try adding additional columns of data or more data and add this view to the Grafana dashboards.